

Plan de tests & jeu d'essai

Application web « Petit Rayon de Soleil ». Plan de tests couvrant les quatre niveaux — unitaires, intégration, système et acceptation utilisateur — avec preuve d'exécution automatisée.

Projet petit-rayon-de-soleil v1.0.0 **Suites** tests/auth.unit.test.js · tests/api.test.js

Commande npm test **Date d'exécution** 13/07/2026

14 / 14

Tous les tests passent.

7 unitaires + 7 d'intégration · 0 échec · 0 ignoré · durée totale 4 201 ms

01 Stratégie et niveaux de test

Le plan couvre les **quatre niveaux de test** attendus, du plus fin (une fonction isolée) au plus global (le parcours utilisateur complet). Les niveaux unitaire et intégration sont **automatisés** (14 tests, module natif `node:test`) ; les niveaux système et acceptation sont validés par ces tests de bout en bout complétés d'une vérification manuelle des parcours.

UNITAIRE

Fonctions isolées

Hachage de mot de passe, extraction du token — sans base ni serveur.

7 tests auto.

INTÉGRATION

Couches assemblées

Route → logique → base SQLite, via de vraies requêtes HTTP.

7 tests auto.

SYSTÈME

Application entière

Serveur réel démarré, parcours métier complet proposition → approbation.

✓ couvert

ACCEPTATION

Besoins utilisateur

Chaque exigence Must du cahier des charges vérifiée côté usage.

✓ couvert

Outil retenu : le module natif `node:test` (assertions `node:assert`), sans aucune dépendance de test à installer — cohérent avec la philosophie minimale du projet et exécutable partout où Node est présent.

02 Environnement de test

Les tests s'exécutent en local. Les **tests unitaires** n'ont besoin d'aucune base. Les **tests d'intégration** travaillent sur une base **réinitialisée à neuf** : le hook `before()` supprime le fichier `soleil.db` puis le recrée et le remplit (seed) avant la première assertion ; le serveur écoute sur un port dédié `3999` pour ne pas entrer en conflit avec une instance de développement.

Système	Windows 10 (win32 x64)
Runtime	Node.js 18+
Base de données	SQLite / libSQL via @libsql/client
Port de test	3999 (isolé du port applicatif 3000)
Framework de test	node:test + node:assert (natif)
Jeu de données	seed : 1 admin, 1 utilisateur, 5 messages

Procédure d'exécution

- 1 Installer les dépendances applicatives : `npm install`.
- 2 S'assurer qu'aucune instance du serveur ne tient `soleil.db` ouverte (sinon le verrou de fichier empêche la réinitialisation de la base sous Windows).
- 3 Lancer la suite complète : `npm test` (raccourci de `node --test`) — les deux fichiers de tests sont découverts automatiquement.

03 Tests unitaires TESTS/AUTH.UNIT.TEST.JS

Ils vérifient des **fonctions pures** de `auth.js`, isolées de toute base de données. Ce sont les briques de sécurité les plus critiques : le hachage des mots de passe et la lecture du jeton d'authentification.

7 tests unitaires — comportement attendu vs obtenu.

#	Fonction testée	Ce qui est vérifié	Attendu	Obtenu
1	<code>hashPassword</code>	Le hash ne contient jamais le mot de passe en clair	<code>hash ≠ clair</code> , format <code>bcrypt</code>	✓ OK
2	<code>checkPassword</code>	Le bon mot de passe est accepté	<code>true</code>	✓ <code>true</code>
3	<code>checkPassword</code>	Un mauvais mot de passe est refusé	<code>false</code>	✓ <code>false</code>
4	<code>hashPassword</code>	Deux hachages du même mot différent (sel aléatoire)	<code>hash1 ≠ hash2</code>	✓ différents
5	<code>tokenFromReq</code>	Extraction correcte d'un en-tête <code>Bearer</code>	le <code>token</code>	✓ OK
6	<code>tokenFromReq</code>	Renvoie <code>null</code> si aucun en-tête	<code>null</code>	✓ <code>null</code>
7	<code>tokenFromReq</code>	Ignore un schéma non-Bearer (ex. <code>Basic</code>)	<code>null</code>	✓ <code>null</code>

04 Tests d'intégration, système & acceptation TESTS/API.TEST.JS

Ces tests démarrent l'application Express réelle et envoient de vraies requêtes HTTP via `fetch`. Ils valident donc à la fois l'**intégration** des couches (route → métier → base), le fonctionnement **système** de l'application entière, et l'**acceptation** des exigences Must du cahier des charges. La colonne « Niveau » précise l'apport dominant de chaque test.

7 tests de bout en bout — niveau, entrée, résultat attendu, résultat obtenu (exécution du 13/07/2026).

#	Scénario	Niveau	Requête	Attendu	Obtenu
1	Message aléatoire (accueil)	INTÉGR.	GET /api/messages/random	200 + contenu non vide	✓ 200
2	Inscription puis connexion	ACCEPT.	POST /api/register → POST /api/login	201 puis 200, jeton émis	✓ 201 / 200
3	Connexion, mauvais mot de passe	SÉCURITÉ	POST /api/login	401 (refus)	✓ 401
4	Proposition sans authentification	SÉCURITÉ	POST /api/messages	401 (non authentifié)	✓ 401
5	Accès admin par un utilisateur normal	SÉCURITÉ	GET /api/admin/pending	403 (interdit)	✓ 403
6	Parcours proposition → approbation	SYSTÈME	POST /api/messages → POST /api/admin/approve/:id	invisible avant, 200, visible après	✓ Conforme
7	Suppression de compte (RGPD)	ACCEPT.	DELETE /api/me	200, reconnexion impossible (401)	✓ 200 / 401

05 Preuve d'exécution — sortie du terminal

Capture fidèle de la sortie de `npm test` le 13/07/2026. Les 14 tests (7 unitaires préfixés [unitaire] + 7 d'intégration) passent sans échec.

petit-rayon-de-soleil – npm test

```
$ npm test
```

```
> petit-rayon-de-soleil@1.0.0 test
> node --test
```

Base initialisée avec un admin, un utilisateur et 5 messages.

```
✓ GET /api/messages/random renvoie un message (1917.03ms)
✓ inscription puis connexion fonctionnent (346.25ms)
✓ connexion avec mauvais mot de passe est refusée (130.44ms)
✓ proposer un message sans être connecté est refusé (401) (4.65ms)
✓ un utilisateur normal ne peut pas accéder à l'admin (403) (134.60ms)
✓ parcours complet : proposition -> approbation admin (290.95ms)
✓ suppression de compte (RGPD) efface l'utilisateur (142.59ms)
✓ [unitaire] hashPassword ne renvoie jamais le mot de passe en clair (151.31ms)
✓ [unitaire] checkPassword accepte le bon mot de passe (289.42ms)
✓ [unitaire] checkPassword refuse un mauvais mot de passe (340.37ms)
✓ [unitaire] deux hachages du même mot de passe sont différents (sel aléatoire) (272.88ms)
✓ [unitaire] tokenFromReq extrait le token d'un en-tête Bearer (0.38ms)
✓ [unitaire] tokenFromReq renvoie null sans en-tête Authorization (0.20ms)
✓ [unitaire] tokenFromReq ignore un schéma non-Bearer (0.20ms)
```

```
i tests 14
```

```
i suites 0
```

```
i pass 14
```

```
i fail 0
```

```
i skipped 0
```

```
i duration_ms 4201.40
```

06 Jeu d'essai détaillé — fonctionnalité la plus représentative

La fonctionnalité retenue est le **parcours proposition** → **approbation** (test système #6). Elle traverse toutes les couches de l'application et implique les deux rôles (utilisateur et administrateur), ce qui en fait le scénario le plus démonstratif du bon fonctionnement du système.

Décomposition étape par étape : entrée → attendu → obtenu → écart.

Étape	Entrée	Attendu	Obtenu	Écart
1 · Proposer	Utilisateur de démo <code>user@soleil.fr</code> connecté envoie <code>POST /api/messages</code> avec <code>{ content: "Tu es capable de grandes choses." }</code>	201, message créé avec <code>approved = 0</code>	✓ 201	—
2 · Vérifier l'invisibilité	<code>GET /api/messages</code> (liste publique)	le message n'apparaît pas	✓ absent	—
3 · Approuver	Admin connecté envoie <code>POST /api/admin/approve/:id</code>	200, passage à <code>approved = 1</code>	✓ 200	—
4 · Vérifier la visibilité	<code>GET /api/messages</code> à nouveau	le message apparaît désormais	✓ présent	—

Aucun écart constaté. Le comportement observé correspond exactement au comportement attendu à chaque étape. Le cycle de modération fonctionne : un message reste privé tant qu'un administrateur ne l'a pas approuvé.

Document généré à partir de l'exécution réelle des suites `tests/auth.unit.test.js` et `tests/api.test.js` · Petit Rayon de Soleil · Titre professionnel CDA.